CHAPTER 1

Introduction to C

You already know how to write programs in Python. In the next few chapters, you are going to learn C. Your Python knowledge will make this easier, because C has the same fundamental ideas: variables, functions, loops, and conditionals. The syntax looks different, and there are a few genuinely new concepts, but you will recognize the shape of everything you write.

Why learn C? Two reasons. First, C is used wherever performance matters: operating systems, game engines, embedded systems, and scientific computing. Second, and more important for this course, C forces you to think about memory explicitly. Python quietly manages memory on your behalf; C makes you do it yourself. Once you understand how C uses memory, you will also understand what Python is doing underneath, and you will be ready to implement data structures from scratch.

1.1 Compiled vs. Interpreted

When you run a Python program, the Python interpreter reads your `.py` file and executes it line by line. There is no separate step between writing and running.

C works differently. Before a C program can run, a *compiler* must translate it into machine code, the raw instructions that your CPU executes directly. The compiler reads your `.c` file, checks it for errors, and produces an *executable* file. You then run that executable.

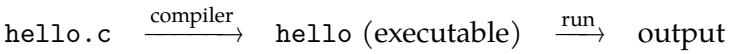


Figure 1.1: The compile-then-run cycle

Because the compiler sees your entire program before anything runs, it can catch many mistakes, like using a variable before declaring it or passing the wrong type to a function, that Python would only discover at runtime.

To compile a C file called `hello.c`, you run:

```
1 gcc hello.c -o hello
```

The `-o hello` flag names the output executable `hello`. Then you run it:

```
./hello
```

1.2 Your First C Program

Create a file called `hello.c` and type the following:

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Compile and run it. You should see:

```
Hello, world!
```

Let's look at each piece.

`#include <stdio.h>` tells the compiler to include the standard input/output library, which provides `printf` for printing. Think of it like Python's `import` statement.

`int main()` is the function where your program starts. Every C program must have exactly one `main` function. The `int` means it returns an integer; `return 0` signals to the operating system that the program finished successfully.

`printf("Hello, world!\n");` prints a line of text. The `\n` is the newline character.

Notice the semicolons at the end of statements, and the curly braces `{ }` around the body of `main`. In Python, indentation defines blocks; in C, curly braces do.

1.3 Types and Variables

In Python, you write:

```python
x = 5
y = 3.14
```

Python figures out the type of each variable automatically.

In C, you must declare the type of every variable when you create it:

```c
int x = 5;
double y = 3.14;
```

This is called *static typing*. Once you declare a variable as `int`, it can only hold integers. The compiler enforces this and will refuse to compile code that mixes types incorrectly.

The common C types are:

`int` A whole number: `0`, `42`, `-7`. Typically 4 bytes.

`double` A floating-point number: `3.14`, `-0.001`. Typically 8 bytes. (There is also `float`, which uses 4 bytes and has less precision.)

`char` A single character: `'A'`, `'z'`, `'7'`. Use single quotes.

`int` Booleans in C are just integers: `0` means false, and any non-zero value means true. You can include `<stdbool.h>` to get the names `true` and `false`.

You can also declare a variable without immediately giving it a value:

```c
int count;      /* declared but not initialized */
count = 10;     /* now it has a value */
```

Warning: an uninitialized variable in C contains whatever random bytes happen to be in that memory location. Unlike Python, C will not raise an error if you read an uninitialized variable; it will just give you garbage. Always initialize your variables.

Note that C uses `/* ... */` for comments, not `#`. You can also use `//` for single-line comments in modern C.

1.4 Printing with `printf`

`printf` uses format specifiers to print different types:

```c
int age = 17;
double height = 1.75;
printf("Age: %d, Height: %f\n", age, height);
```

Output:

```
Age: 17, Height: 1.750000
```

Common format specifiers:

Specifier	Type
%d	int
%f	double or float
%c	char
%s	string (char pointer)

1.5 Conditionals and Loops

The logic is the same as Python; only the syntax changes.

1.5.1 if / else

```c
int score = 85;

if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else {
    printf("C or below\n");
}
```

The condition must be in parentheses `( )`, and each branch is surrounded by curly braces `{ }`.

1.5.2 for and while loops

```c
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

A C `for` loop has three parts separated by semicolons: the initializer (`int i = 0`), the

condition (`i < 5`), and the update (`i++`). The `++` increments by 1, the same as `i = i + 1`.

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

1.6 Functions

In C, you must declare the return type and the type of each parameter:

```c
double square(double x)
{
    return x * x;
}

int main()
{
    printf("%f\n", square(4.0));   /* prints 16.000000 */
    printf("%f\n", square(2.5));   /* prints 6.250000  */
    return 0;
}
```

If a function returns nothing, its return type is `void`:

```c
void printGreeting(char *name)
{
    printf("Hello, %s!\n", name);
}
```

Functions must be defined *before* they are called, or you must provide a *forward declaration* (just the signature, ending in a semicolon) above `main`:

```c
double square(double x);   /* forward declaration */

int main()
{
    printf("%f\n", square(3.0));
    return 0;
}

```

```c
double square(double x)    /* full definition can come later */
{
    return x * x;
}
```

1.7 The Stack: Blackboards for Functions

Every function, while it is running, needs somewhere to store its local variables. C gives each function a *frame*: a reserved chunk of memory that holds all of that function's local variables and parameters.

Think of a frame as a **blackboard**. When a function starts running, it gets a fresh blackboard. It can write anything it wants on that blackboard, create variables, change their values. When the function finishes, the blackboard is *erased and discarded*. Local variables do not survive after their function returns.

Where do these frames live? In a region of memory called the *stack*. The stack works exactly like a physical stack of objects: the most recently added item is always on top. When `main` calls a function, that function's frame is pushed onto the top of the stack. When the function returns, its frame is popped off, and `main`'s frame is on top again.

Here is a concrete example:

```c
#include <stdio.h>

int cookingMinutes(int weightPounds)
{
    int minutes = 15 + 15 * weightPounds;
    return minutes;
}

int main()
{
    int totalWeight = 10;
    int gibletsWeight = 1;
    int turkeyWeight = totalWeight - gibletsWeight;
    int result = cookingMinutes(turkeyWeight);
    printf("Cook for %d minutes.\n", result);
    return 0;
}
```

When `main` calls `cookingMinutes(9)`, the stack looks like this:

cookingMinutes()	`weightPounds = 9` `minutes = 150`

main()	`totalWeight = 10`
	`gibletsWeight = 1`
	`turkeyWeight = 9`
	`result = ?`

`main`'s blackboard sits below; `cookingMinutes`'s blackboard sits on top of it. The variable `minutes` inside `cookingMinutes` is completely separate from anything in `main`. Each function only sees its own blackboard.

When `cookingMinutes` returns 150, its frame is discarded. `main`'s frame resumes, and `result` is filled in with 150.

1.7.1 Parameters are copies

When you call `cookingMinutes(turkeyWeight)`, C copies the value of `turkeyWeight` into the new parameter `weightPounds` on `cookingMinutes`'s blackboard. Changing `weightPounds` inside the function would not change `turkeyWeight` in `main`. They are separate variables. This is called *pass-by-value*.

1.8 Addresses and Pointers

Every variable in your program lives at some address in memory. You can find out the address of a variable using the `&` operator, called the *address-of* operator:

```c
#include <stdio.h>

int main()
{
    int i = 17;
    printf("i stores its value at %p\n", (void *)&i);
    return 0;
}
```

Run this and you will see something like:

```
i stores its value at 0x7ffeea3c4738
```

The address is printed in hexadecimal. Your address will differ, because the operating

system decides where in memory your variables live.

A *pointer* is a variable that holds an address. If you want to store the address of an `int`, you declare a variable of type `int *` ("pointer to int"):

```c
int i = 17;
int *addressOfI = &i;
printf("i is at %p\n", (void *)addressOfI);
```

The `*` in the declaration is part of the type; it means "this variable holds an address, and the thing at that address is an `int`."

1.8.1 Reading and writing through a pointer

Once you have a pointer, you can use the `*` operator to read or write the value *at* the address the pointer holds. This is called *dereferencing* the pointer:

```c
int i = 17;
int *p = &i;

printf("%d\n", *p);   /* prints 17 -- reads through p  */

*p = 89;              /* writes 89 to the address p holds */
printf("%d\n", i);    /* prints 89 -- i was changed!   */
```

To summarize:

- `&i` gives you the address of `i`.
- `int *p` declares a variable that can hold such an address.
- `*p` gives you the value stored at the address `p` holds.

1.8.2 NULL pointers

Sometimes you want a pointer that does not yet point to anything. Use `NULL`:

```c
int *p = NULL;   /* p points to nothing */

if (p) {
    printf("%d\n", *p);     /* only runs if p is non-null */
```

```
} else {
    printf("p is null\n");
}
```

A null pointer is simply the address zero. **Never dereference a null pointer**, your program will crash. Always check before dereferencing a pointer that might be null.

1.9 Pass-by-Reference

Recall that when you call a function in C, arguments are *copied* into the function's frame. But what if you want a function to modify a variable in the calling function? You pass the *address* of that variable instead of its value. The function can then write to that address.

There is a standard C function called `modf()` that splits a floating-point number into its integer part and its fractional part, and needs to return *both*. Since a function can only return one value, it returns the fractional part and writes the integer part to an address you supply:

```
#include <stdio.h>
#include <math.h>

int main()
{
    double pi = 3.14;
    double integerPart;
    double fractionPart;

    /* Pass the address of integerPart so modf can write there */
    fractionPart = modf(pi, &integerPart);

    printf("Integer part:  %f\n", integerPart);
    printf("Fraction part: %f\n", fractionPart);
    return 0;
}
```

Output:

```
Integer part:  3.000000
Fraction part: 0.140000
```

Here is another way to think about it. Imagine you need to give a spy an assignment. You say: "I've left a length of steel pipe at the foot of the angel statue in the park. When you have the information, roll it up and leave it in the pipe. I'll pick it up Tuesday." In the spy

business, this is called a *dead drop*.

Pass-by-reference works the same way. You are not giving the function the data directly, you are giving it a *location* where it can leave the result.

Here is our own pass-by-reference function that converts meters to feet and inches:

```c
#include <stdio.h>
#include <math.h>

void metersToFeetAndInches(double meters,
                           int *ftPtr,
                           double *inPtr)
{
    double rawFeet = meters * 3.281;
    int feet = (int)floor(rawFeet);
    double inches = (rawFeet - feet) * 12.0;

    if (ftPtr) *ftPtr = feet;
    if (inPtr) *inPtr = inches;
}

int main()
{
    int feet;
    double inches;
    metersToFeetAndInches(1.8, &feet, &inches);
    printf("%d ft %f in\n", feet, inches);
    return 0;
}
```

The caller declares the variables it wants filled in (`feet`, `inches`), passes their addresses to the function, and the function writes the results there.

1.10 Structs: Grouping Related Data

A *struct* lets you bundle several related values into a single named type. In Python you might use a dictionary for this; in C, a struct is the fundamental grouping mechanism.

```c
struct Person {
    double heightInMeters;
    int weightInKilos;
};
```

This defines a new type called `struct Person`. You can then create variables of that type:

```c
#include <stdio.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person p)
{
    return p.weightInKilos / (p.heightInMeters * p.heightInMeters);
}

int main()
{
    struct Person mikey;
    mikey.heightInMeters = 1.70;
    mikey.weightInKilos  = 96;

    struct Person aaron;
    aaron.heightInMeters = 1.97;
    aaron.weightInKilos  = 84;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));
    printf("aaron BMI: %f\n", bodyMassIndex(aaron));
    return 0;
}
```

You access the members of a struct using a dot: `mikey.heightInMeters`.

When `main` is running, the stack looks like this:

main()	`mikey.heightInMeters = 1.70`
	`mikey.weightInKilos = 96`
	`aaron.heightInMeters = 1.97`
	`aaron.weightInKilos = 84`

The two `Person` structs live right inside `main`'s frame on the stack.

1.11 The Heap

The stack is fast and automatic, but it has two constraints: stack frames are destroyed when functions return, and the sizes of all stack-allocated variables must be known at compile time.

The *heap* is a separate region of memory that has neither constraint. Memory on the heap

persists until you explicitly release it, and can be any size determined at runtime.

In C, you claim heap memory with `malloc` and release it with `free`.

```c
#include <stdlib.h>

struct Person *mikey = malloc(sizeof(struct Person));
```

`malloc` asks the system for enough heap memory to hold a `Person` struct and returns the address of that memory. The variable `mikey` is a *pointer* to a `Person`, it lives on the stack, but the struct it points to lives on the heap.

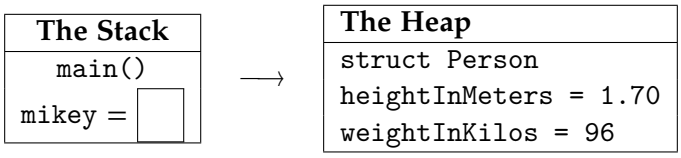


Figure 1.2: A pointer on the stack pointing to a struct on the heap

1.11.1 The arrow operator

When you have a pointer to a struct, you access its members using `->` instead of a dot:

```c
mikey->heightInMeters = 1.70;
mikey->weightInKilos  = 96;
```

The code `mikey->weightInKilos` means: "dereference the pointer `mikey` to get the struct, then access its `weightInKilos` member." It is shorthand for `(*mikey).weightInKilos`.

1.11.2 Releasing heap memory

When you are done with heap-allocated memory, you must release it with `free`:

```c
free(mikey);
mikey = NULL;   /* good habit: prevents accidental reuse */
```

After `free`, the memory is returned to the heap for reuse. Setting the pointer to `NULL` prevents you from accidentally using it again.

If you forget to call `free`, the memory is never returned. This is called a *memory leak*. In a short-lived program it may not matter, but in a long-running program it can consume all available memory.

1.11.3 A complete heap example

```c
#include <stdio.h>
#include <stdlib.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person *p)
{
    return p->weightInKilos / (p->heightInMeters * p->heightInMeters);
}

int main()
{
    struct Person *mikey = malloc(sizeof(struct Person));
    mikey->heightInMeters = 1.70;
    mikey->weightInKilos  = 96;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));

    free(mikey);
    mikey = NULL;

    return 0;
}
```

`bodyMassIndex` now takes a `struct Person *` instead of a `struct Person` by value. This avoids copying the struct. Functions that work on heap-allocated data almost always take pointers.

1.11.4 Heap arrays

You can also allocate a contiguous block of memory for multiple values. Use `malloc` with a count:

```c
/* Allocate space for 1000 doubles on the heap */
double *buffer = malloc(1000 * sizeof(double));
```

```
3
4   buffer[0] = 3.14;
5   buffer[1] = 2.72;
6
7   free(buffer);
8   buffer = NULL;
```

A pointer to the first element is all you need to work with the whole array. The heap memory is contiguous, so `buffer[0]`, `buffer[1]`, and so on are laid out one after another.

1.12 What Python Objects Actually Are

You now know everything you need to understand what a Python object is, under the hood.

When Python creates an object, it:

1. Allocates a struct on the heap large enough to hold the object's data (its instance variables).
2. Stores in that struct a pointer to the class's method table, so that method calls can find the right code.
3. Returns a pointer to that struct.

Your Python variable `obj` is, at the machine level, a pointer to a heap-allocated struct. When Python's garbage collector determines that nothing points to an object anymore, it calls the equivalent of `free` to release that memory.

The next chapter introduces C++ classes, and with this memory model in hand, you will implement linked lists, trees, and hash tables from scratch.

1.13 Exercises

1. Write a C function `celsiusToFahrenheit` that takes a `double` temperature in Celsius and returns the equivalent in Fahrenheit. Call it from `main` for the values 0, 20, 37, and 100, and print each result.
2. Write a function `swap` that takes two `int` pointers and swaps the values at the two addresses. Test it so that after calling `swap(&a, &b)`, the values of `a` and `b` have been exchanged.
3. Write a struct `Point` with `double` members `x` and `y`. Write a function `distance` that takes two `struct Point *` pointers and returns the Euclidean distance between

them. Allocate two points on the heap, fill in their coordinates, print the distance, and then free the memory.

4. **Memory leak hunt.** The following program has a memory leak. Identify and fix it.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
};

void printValue(int x)
{
    struct Node *n = malloc(sizeof(struct Node));
    n->value = x;
    printf("%d\n", n->value);
}

int main()
{
    for (int i = 0; i < 5; i++) {
        printValue(i);
    }
    return 0;
}
```

5. A `struct tm` is defined in `<time.h>` and holds a broken-down calendar time. The function `time(NULL)` returns the number of seconds since midnight, January 1, 1970. The function `localtime_r(&seconds, &now)` fills a `struct tm` with the corresponding date and time. Here is a snippet to get you started:

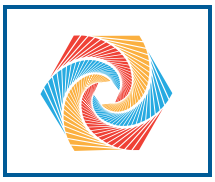
```c
#include <stdio.h>
#include <time.h>

int main()
{
    long secondsSince1970 = time(NULL);
    struct tm now;
    localtime_r(&secondsSince1970, &now);
    printf("The time is %d:%d:%d\n",
           now.tm_hour, now.tm_min, now.tm_sec);
    return 0;
}
```

Look up the other fields of `struct tm` and modify the program to print today's full date. Then modify it to print the date that falls exactly 4,000,000 seconds from now. (Hint: `tm_mon` is 0-indexed, so January is 0.)

This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.

Appendix A

Answers to Exercises



Index